Changelog: Version 0.21.2

CHANGES

- **Protocol Incompatibility:** Version 0.21.2 is **not compatible** with any previous releases due to fundamental changes in the communication protocol and synchronization model.
- **Configuration Incompatibility** Version 0.21.2 is **not compatible** with any configuration created with previous releases due to some changes how the din_t is handled inside the library.
- **Event Naming:** Event names have been standardized across the system. If you are upgrading from v0.20.1, you **must** update the names of your event handlers and listeners to match the new standard.
- **DDO API:** Manually setting the `timestamp` and `origin` fields on a DDO is no longer permitted. These fields are now set automatically and exclusively by the library to ensure system integrity.
- **Incoming Request:** Now it is possible to specify the policy of the incoming request with three different policies. More specifications on reference file.
- **Discovery Activation Modes:** The discovery resolution can be configured into four distinct modes:
  - `discovery_off` : Completely disables discovery resolution.
  - `discovery_sender_only` : The node can actively search for an existing network but will not act as a host or responder if it receives discovery packets from other nodes.
  - `discovery_receiver_only` : The node can host and assign networks to other nodes, but it is restricted from sending discovery beacons to join external networks.
  - `discovery_full` : Enables full bidirectional discovery, allowing the node to either create a new network or join an existing one.
- **DDO retry on failure policies**: It is now possible to configure how the system behaves when a DDO fails to be sent. See the reference file for more details.

New Features

Networking & Discovery

- **Automatic Network Formation:** Nodes can now be initialized without a predefined SID or DIN when the **discovery feature** is enabled.

- **DHCP-like Resolution**: When a DIN is not provided, the node automatically generates one using a time-based algorithm, similar to how DHCP assigns addresses. The SID, however, is always derived from the master node's SID when joining a network. If initialization has not been performed, the node cannot communicate until it joins a network via the discovery mechanism. Therefore, a node without a SID and DIN can only perform discovery to attach to an existing network; once connected, it can begin normal communication. Note: At least one nearby node must already have a configured SID, otherwise no network can be created.
- **Beaconing:** The discovery function (available to end-users and wrappers) allows a node to broadcast beacons. Other nodes in range can detect and map it dynamically, enabling networks to self-organize without user interaction.
- **Security:** Nodes can be configured to ignore or reject incoming discovery requests for security and privacy.

Locate

- **Custom Timeout:** The `node.locate()` function now accepts an optional second parameter to define a **custom timeout** in milliseconds. The default remains `1000ms`. This is crucial for adapting to different network latencies.

Usage 1	Usage 2 (100ms timeout)
`node.locate(din);`	`node.locate(din, 100);`

- **Flexible Location:** Nodes can now be located using either their full DIN (`user_part + network_creation_time`) or just the user part.

DDO (Dynamic Data Object)

- **Payload-less DDOs:** The system now permits sending DDOs without a payload. This is useful for lightweight control messages, acknowledgments, or status signals, reducing network overhead.

Changes & Enhancements

Node Activation & Synchronization

- **Mandatory Synchronization:** Nodes now require a preliminary synchronization phase via the **ATS protocol** before bidirectional communication is allowed. This process is initiated when a DDO is pushed to a newly added node or when connecting to a network.
- **SID Management:** To solve communication conflicts, a node's SID is now assigned by the network's master node upon discovery. This will override any SID set by the user, ensuring all nodes in a network can communicate.

ATS

- **Master Selection Logic:** The dATS system now assigns priority and designates the **node that receives the discovery packet** as the master. A node's power-on time is no longer relevant for this decision.
- **Network Integration:** A master is only set when a *new* network is created. If a node joins an *existing* network, it will follow the synchronization dictated by the current master, ensuring a single, stable time reference.
- **Better dATS algorithm** for a faster and more precise sync.

Protocol (Internal Enhancements)

- **Dynamic DME Parsing:** The internal protocol has been significantly enhanced to process packet headers and payloads independently.
- **Improved Security & Efficiency:** The header is decoded and validated first. If the header is invalid, the payload is discarded *without* being processed, saving memory and providing a defense against resource-exhaustion DoS attacks.
- **High-Precision Timestamping:** This separation allows for distinct timestamps for header arrival (critical for ATS) and full payload decoding (used for `dperf` benchmarks), resolving a key source of timing inaccuracy.
- **Flexible Packet Structure:** The DME can now have a non-fixed structure, allowing variable-format packets to be transmitted, which optimizes memory usage.

Enhanced Driver Management

- **Connection Monitoring & Failover:** Drivers can now detect remote connection failures. Upon detection, the system automatically attempts to reach the remote node using alternative known network paths, ensuring continuous connectivity.
- **ACK DME Statistics (Internal only):** A new "ACK DME" message type has been introduced, allowing nodes to gather critical transmission metrics. The system analyzes channel latency and ranks available drivers/channels accordingly, ensuring that the most efficient communication path is always prioritized.

Improved System Stability

- **Event-Callback Recovery:** DaaS now includes a dedicated event-callback mechanism for system recovery in the event of module failure.
- **Atomic Rollback:** During critical operations such as **dATS synchronization** and **Networking**, the system monitors for errors. If a failure occurs, it can revert to the last known functional state by rolling back all changes made during the failed operation, preventing system corruption.

Added support to: ESP32 RISCV and RASPBERRY (ARM6/7/8)

Known Limitations

- **PERFORM_CORE_NO_THREAD with LOCATE:** When using locate with the perform mode `PERFORM_CORE_NO_THREAD` the locate functionality might fail if the node is not present inside the routing table and it has to be searched. This occurs since the threadless mode, doesn't not operate during the blocking waiting. So at the first try it might fail but it should work on the further ones. It is adviced to implement inside the locate function of the wrappers a logic like this:

  ```
  for (int delay_s = 1; delay_s <= 10; delay_s ++){
      daas_error_t locate_result = node->locate(remoteDin, 100);

      // Since this node operates without threads, we need to perform the node's task
      if (perform_mode == PERFORM_CORE_NO_THREAD)
          node->doPerform(PERFORM_CORE_NO_THREAD);

      if (locate_result == ERROR_NONE)
          return ERROR_NONE;
  }

  return ERROR_DIN_UNKNOWN;
  ```

  NOTE: this does not apply if the mode of the doPerform is set to `PERFORM_CORE_THREAD` . It is adviced to store the current node perform setting to apply the above logic.

- **Discovery Failures on First Attempt:** On low-end hardware, the discovery procedure may occasionally fail mid-way. If this occurs, it is advised to wait a few seconds before re-transmitting discovery beacons. In most cases, the second attempt successfully completes the node discovery.

- **Self-Targeted DME Restriction:** Due to recent architectural changes in how din is handled, sending packets where the destination is the local node (self-sending) is currently non-functional.